PYTHON ACTIVITY-BASED LEARNING

10 Selected Case Studies

Problem Statements • Objectives • Assumptions • Algorithms • Flowcharts • Pseudocode • Python Source Code • Test Cases • Sample Outputs • Conclusions

Name	Navadeep Akkinapalli
Roll Number	26001438
Branch	CSD
Section	2

Selected from the supplied Python Activity-Based Learning case-study list. The programs use introductory Python concepts and simplified assumptions for classroom practice.

Contents

Case Study	Title
4	Employee Payroll and Deduction Analysis
9	Multi-Currency Travel Budget Analysis
11	Examination Result and Grace-Mark Validation
12	Attendance Recovery Computation
13	ATM Transaction Sequence Simulation
14	Warehouse Inventory Reconciliation
17	Student Performance Trend Analysis
18	Fuel Consumption and Route Comparison
27	Advanced Numeric Pattern Generator
29	Password Policy Validation Engine

Note: The flowcharts in this document are text-based flowcharts that show the main processing steps and decisions. The sample outputs are formatted console-output examples, not captured screenshots from a live Python interpreter.

Case Study 4: Employee Payroll and Deduction Analysis

1. Problem Statement

Write a Python program to calculate an employee's gross salary, deduction, and net salary using basic salary, House Rent Allowance (HRA), and Dearness Allowance (DA). For this simplified activity, the deduction is 10% of gross salary.

2. Objectives

• Accept basic salary, HRA, and DA from the user.

• Calculate gross salary and a 10% deduction.

• Display the net salary clearly.

3. Assumptions

• All salary inputs are non-negative numbers.

• Gross salary is the sum of basic salary, HRA, and DA.

• Deduction is fixed at 10% of gross salary for this exercise.

4. Algorithm

1. Start.

2. Read basic salary, HRA, and DA.

3. Calculate gross = basic salary + HRA + DA.

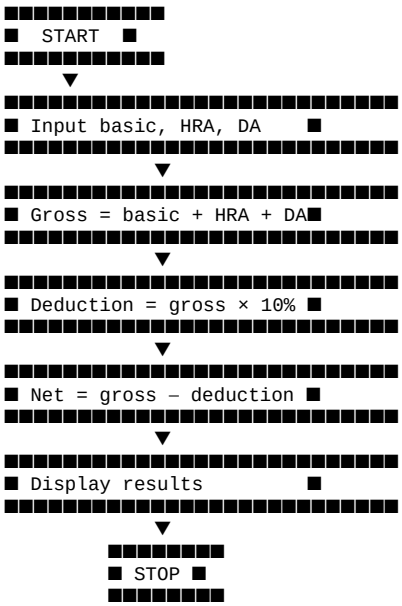
4. Calculate deduction = gross × 0.10.

5. Calculate net salary = gross – deduction.

6. Display gross salary, deduction, and net salary.

7. Stop.

5. Flowchart



6. Pseudocode

```
BEGIN
    INPUT basic, hra, da
    gross = basic + hra + da
    deduction = gross * 0.10
    net = gross - deduction
    PRINT gross, deduction, net
```

```
END
```

7. Python Source Code

```
basic = float(input("Enter basic salary: "))
hra = float(input("Enter HRA: "))
da = float(input("Enter DA: "))

gross = basic + hra + da
deduction = gross * 0.10
net = gross - deduction

print("Gross Salary:", gross)
print("Deduction:", deduction)
print("Net Salary:", net)
```

8. Test Cases

Basic	HRA	DA	Expected result
20000	5000	3000	Gross 28000; deduction 2800; net 25200
10000	1000	500	Gross 11500; deduction 1150; net 10350

9. Sample Output

```
Enter basic salary: 20000
Enter HRA: 5000
Enter DA: 3000
Gross Salary: 28000.0
Deduction: 2800.0
Net Salary: 25200.0
```

10. Conclusion

The program calculates gross salary, a fixed 10% deduction, and net salary using arithmetic operators and variables.

Case Study 9: Multi-Currency Travel Budget Analysis

1. Problem Statement

Write a program that converts a travel expense in a foreign currency into Indian rupees using a supplied exchange rate, then checks whether the converted expense is within the user's budget.

2. Objectives

• Read an exchange rate, foreign expense, and rupee budget.

• Convert the expense into INR.

• Compare the converted amount with the budget.

3. Assumptions

• The exchange rate is the number of Indian rupees for one unit of foreign currency.

• Inputs are non-negative and the exchange rate is greater than zero.

• Conversion fees are not included in this simplified program.

4. Algorithm

1. Start.

2. Read exchange rate, foreign expense, and budget.

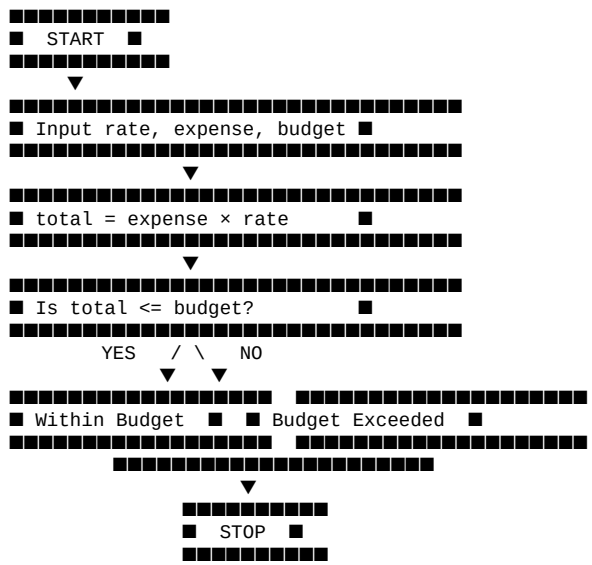
3. Calculate total INR expense = foreign expense × exchange rate.

4. Display the converted expense.

5. If total expense is less than or equal to budget, display Within Budget; otherwise display Budget Exceeded.

6. Stop.

5. Flowchart



6. Pseudocode

```
BEGIN
    INPUT rate, expense, budget
    total = expense * rate
    PRINT total
    IF total <= budget THEN
        PRINT "Within Budget"
    ELSE
        PRINT "Budget Exceeded"
    ENDIF
END
```

7. Python Source Code

```
rate = float(input("Enter exchange rate: "))
expense = float(input("Enter travel expense in foreign currency: "))
budget = float(input("Enter budget in rupees: "))

total = expense * rate

print("Total Expense in INR:", total)

if total <= budget:
    print("Within Budget")
else:
    print("Budget Exceeded")
```

8. Test Cases

Rate	Expense	Budget	Expected result
90	100	10000	INR 9000; Within Budget
90	150	10000	INR 13500; Budget Exceeded

9. Sample Output

```
Enter exchange rate: 90
Enter travel expense in foreign currency: 100
Enter budget in rupees: 10000
Total Expense in INR: 9000.0
Within Budget
```

10. Conclusion

The program converts a foreign expense to INR and uses an if-else statement to check the travel budget.

Case Study 11: Examination Result and Grace-Mark Validation

1. Problem Statement

Write a Python program to read marks in three subjects, calculate total and average marks, and decide whether the student passes. This simplified version checks the pass mark only and does not award grace marks.

2. Objectives

• Read marks in three subjects.

• Calculate total and average marks.

• Check whether every subject meets the minimum pass mark.

3. Assumptions

• Each subject is marked out of 100.

• The pass mark is 35 in every subject.

• Grace marks are not applied in this simplified solution.

4. Algorithm

1. Start.

2. Read marks for three subjects.

3. Calculate total = m1 + m2 + m3.

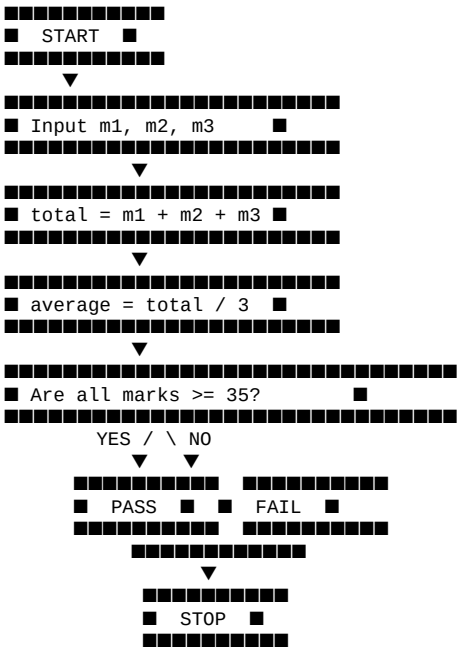
4. Calculate average = total / 3.

5. If all marks are at least 35, display PASS; otherwise display FAIL.

6. Display total and average.

7. Stop.

5. Flowchart



6. Pseudocode

```
BEGIN
    INPUT m1, m2, m3
    total = m1 + m2 + m3
    average = total / 3
```

```
    PRINT total, average
    IF m1 >= 35 AND m2 >= 35 AND m3 >= 35 THEN
        PRINT "PASS"
    ELSE
        PRINT "FAIL"
    ENDIF
END
```

7. Python Source Code

```
m1 = int(input("Enter marks in Subject 1: "))
m2 = int(input("Enter marks in Subject 2: "))
m3 = int(input("Enter marks in Subject 3: "))

total = m1 + m2 + m3
average = total / 3

print("Total Marks:", total)
print("Average:", average)

if m1 >= 35 and m2 >= 35 and m3 >= 35:
    print("Result: PASS")
else:
    print("Result: FAIL")
```

8. Test Cases

Subject 1	Subject 2	Subject 3	Expected result
70	80	60	Total 210; average 70; PASS
70	30	60	Total 160; average 53.33; FAIL

9. Sample Output

```
Enter marks in Subject 1: 70
Enter marks in Subject 2: 80
Enter marks in Subject 3: 60
Total Marks: 210
Average: 70.0
Result: PASS
```

10. Conclusion

The program uses arithmetic operators and a compound if condition to calculate results and determine pass or fail status.

Case Study 12: Attendance Recovery Computation

1. Problem Statement

Write a Python program to calculate attendance percentage and decide whether a student is eligible for an examination when the minimum attendance requirement is 75%.

2. Objectives

• Read the total number of classes and classes attended.

• Calculate attendance percentage.

• Display eligibility using a conditional statement.

3. Assumptions

• Total classes must be greater than zero.

• Attended classes are between zero and total classes.

• The eligibility threshold is 75%.

4. Algorithm

1. Start.

2. Read total classes and attended classes.

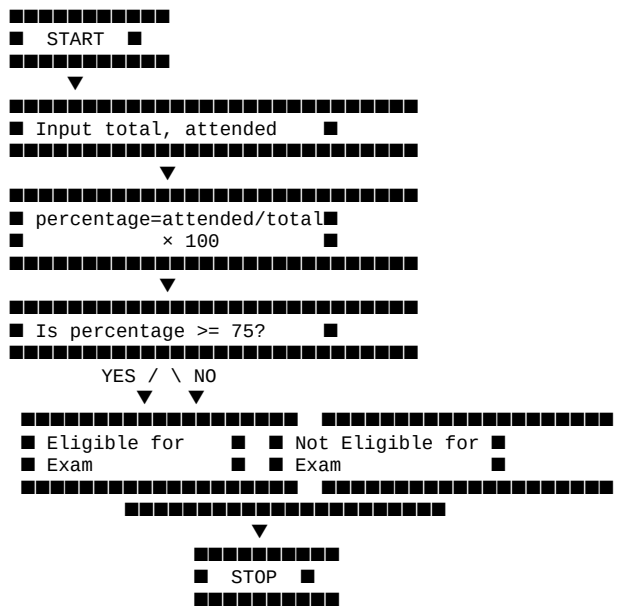
3. Calculate percentage = attended / total × 100.

4. Display the percentage.

5. If percentage is at least 75, display Eligible; otherwise display Not Eligible.

6. Stop.

5. Flowchart



6. Pseudocode

```
BEGIN
    INPUT total, attended
    percentage = (attended / total) * 100
    PRINT percentage
    IF percentage >= 75 THEN
        PRINT "Eligible for Exam"
    ELSE
        PRINT "Not Eligible for Exam"
```

```
    ENDIF
END
```

7. Python Source Code

```
total = int(input("Enter total classes: "))
attended = int(input("Enter attended classes: "))

percentage = (attended / total) * 100

print("Attendance Percentage:", percentage)

if percentage >= 75:
    print("Eligible for Exam")
else:
    print("Not Eligible for Exam")
```

8. Test Cases

Total classes	Attended	Expected result
100	80	80%; Eligible
100	60	60%; Not Eligible

9. Sample Output

```
Enter total classes: 100
Enter attended classes: 80
Attendance Percentage: 80.0
Eligible for Exam
```

10. Conclusion

The program calculates attendance percentage and checks it against a 75% eligibility requirement.

Case Study 13: ATM Transaction Sequence Simulation

1. Problem Statement

Write a menu-based Python program that simulates a basic ATM. The user can deposit money, withdraw money, or check the current balance.

2. Objectives

• Display ATM operations in a menu.

• Perform deposits and valid withdrawals.

• Prevent withdrawals greater than the available balance.

3. Assumptions

• The starting balance is ■5,000.

• Transaction amounts should be positive.

• This simplified version performs one transaction per run.

4. Algorithm

1. Start.

2. Set balance to 5000.

3. Display Deposit, Withdraw, and Check Balance options.

4. Read the user's choice.

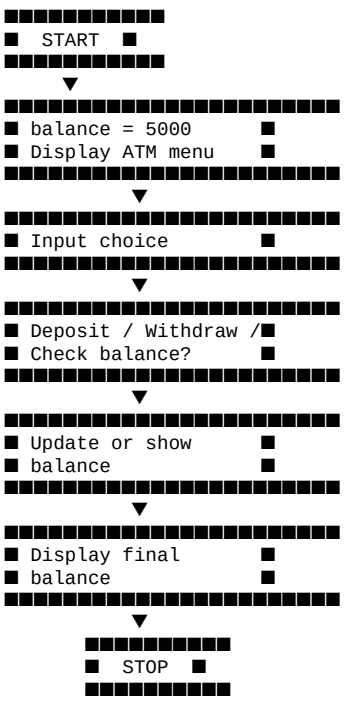
5. If Deposit, read amount and add it to balance.

6. If Withdraw, check amount is positive and not greater than balance; then subtract it.

7. If Check Balance, display the balance; otherwise display invalid choice.

8. Display final balance and stop.

5. Flowchart



6. Pseudocode

```
BEGIN
    balance = 5000
    DISPLAY menu
    INPUT choice
    IF choice = 1 THEN
        INPUT amount
        balance = balance + amount
        PRINT "Amount Deposited"
    ELSE IF choice = 2 THEN
        INPUT amount
        IF amount > 0 AND amount <= balance THEN
            balance = balance - amount
            PRINT "Withdrawal Successful"
        ELSE
            PRINT "Invalid Amount or Insufficient Balance"
        ENDIF
    ELSE IF choice = 3 THEN
        PRINT balance
    ELSE
        PRINT "Invalid Choice"
    ENDIF
    PRINT balance
END
```

7. Python Source Code

```
balance = 5000

print("1. Deposit")
print("2. Withdraw")
print("3. Check Balance")

choice = int(input("Enter your choice: "))

if choice == 1:
    amount = int(input("Enter deposit amount: "))
    if amount > 0:
        balance = balance + amount
        print("Amount Deposited")
    else:
        print("Invalid Amount")

elif choice == 2:
    amount = int(input("Enter withdrawal amount: "))
    if amount > 0 and amount <= balance:
        balance = balance - amount
        print("Withdrawal Successful")
    else:
        print("Insufficient Balance or Invalid Amount")

elif choice == 3:
    print("Balance:", balance)

else:
    print("Invalid Choice")

print("Final Balance:", balance)
```

8. Test Cases

Choice	Amount	Expected result
1	1000	Deposit successful; final balance 6000
2	1000	Withdrawal successful; final balance 4000
2	6000	Withdrawal rejected; balance remains 5000
3	—	Balance displayed as 5000

9. Sample Output

```
1. Deposit
2. Withdraw
```

```
3. Check Balance
Enter your choice: 2
Enter withdrawal amount: 1000
Withdrawal Successful
Final Balance: 4000
```

10. Conclusion

The program demonstrates menu selection, conditional statements, and balance updates for basic ATM transactions.

Case Study 14: Warehouse Inventory Reconciliation

1. Problem Statement

Write a Python program to store product names, opening stock, and sold quantities in lists. Calculate and display the remaining stock for each product.

2. Objectives

• Store product names and quantities using lists.

• Calculate remaining stock for each product.

• Display product-wise stock details using a loop.

3. Assumptions

• The example includes three products.

• Sold quantities do not exceed opening stock.

• Remaining stock equals opening stock minus sold quantity.

4. Algorithm

1. Start.

2. Create lists for product names, opening stock, and sold quantities.

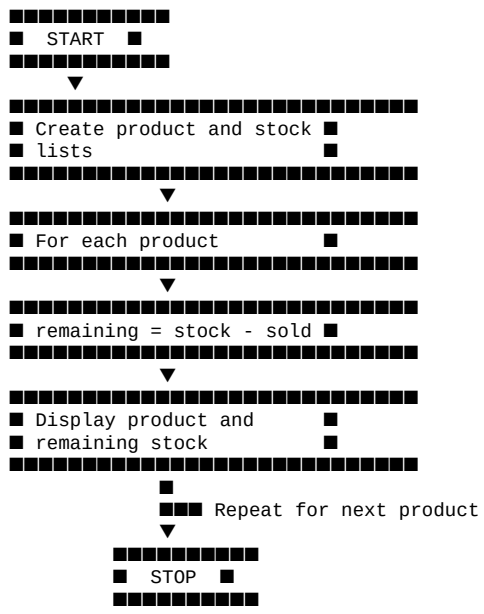
3. Repeat for each product using a loop.

4. Calculate remaining = stock – sold.

5. Display the product name and remaining stock.

6. Stop.

5. Flowchart



6. Pseudocode

```
BEGIN
    products = ["Pen", "Book", "Pencil"]
    stock = [100, 50, 80]
    sold = [20, 10, 30]
    FOR i = 0 TO 2
        remaining = stock[i] - sold[i]
        PRINT products[i], remaining
    ENDFOR
```

```
END
```

7. Python Source Code

```
products = ["Pen", "Book", "Pencil"]
stock = [100, 50, 80]
sold = [20, 10, 30]

for i in range(3):
    remaining = stock[i] - sold[i]

    print("Product:", products[i])
    print("Remaining Stock:", remaining)
```

8. Test Cases

Product	Opening stock	Sold	Expected remaining
Pen	100	20	80
Book	50	10	40
Pencil	80	30	50

9. Sample Output

```
Product: Pen
Remaining Stock: 80
Product: Book
Remaining Stock: 40
Product: Pencil
Remaining Stock: 50
```

10. Conclusion

The program uses lists and a for loop to calculate and display remaining inventory for multiple products.

Case Study 17: Student Performance Trend Analysis

1. Problem Statement

Write a Python program to compare a student's marks in two examinations and report whether performance improved, declined, or stayed the same.

2. Objectives

• Accept marks from two examinations.

• Calculate the difference between the marks.

• Classify the performance trend.

3. Assumptions

• Each examination mark is between 0 and 100.

• The difference is second examination marks minus first examination marks.

• No additional subjects or grading rules are considered.

4. Algorithm

1. Start.

2. Read first and second examination marks.

3. Calculate difference = second marks – first marks.

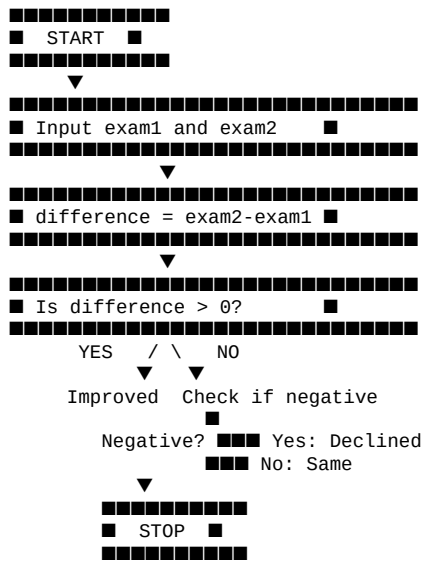
4. If difference is positive, report improvement.

5. If difference is negative, report decline.

6. Otherwise report no change.

7. Stop.

5. Flowchart



6. Pseudocode

```
BEGIN
    INPUT exam1, exam2
    difference = exam2 - exam1
    IF difference > 0 THEN
        PRINT "Performance Improved"
    ELSE IF difference < 0 THEN
        PRINT "Performance Declined"
    ELSE
```

```
        PRINT "Performance Remained the Same"
    ENDIF
END
```

7. Python Source Code

```
exam1 = int(input("Enter first exam marks: "))
exam2 = int(input("Enter second exam marks: "))

difference = exam2 - exam1

print("First Exam Marks:", exam1)
print("Second Exam Marks:", exam2)

if difference > 0:
    print("Performance Improved by", difference, "marks")
elif difference < 0:
    print("Performance Declined by", -difference, "marks")
else:
    print("Performance Remained the Same")
```

8. Test Cases

Exam 1	Exam 2	Expected result
60	80	Improved by 20 marks
80	65	Declined by 15 marks
70	70	Remained the same

9. Sample Output

```
Enter first exam marks: 60
Enter second exam marks: 80
First Exam Marks: 60
Second Exam Marks: 80
Performance Improved by 20 marks
```

10. Conclusion

The program uses subtraction and if-elif-else conditions to identify changes in student performance.

Case Study 18: Fuel Consumption and Route Comparison

1. Problem Statement

Write a Python program to calculate the fuel needed for a journey and estimate the fuel cost from the distance, vehicle mileage, and fuel price.

2. Objectives

• Read journey distance, vehicle mileage, and fuel price.

• Calculate fuel required.

• Calculate and display the total fuel cost.

3. Assumptions

• Distance is measured in kilometres.

• Mileage is measured in kilometres per litre and must be greater than zero.

• Fuel price is given in rupees per litre.

4. Algorithm

1. Start.

2. Read distance, mileage, and price per litre.

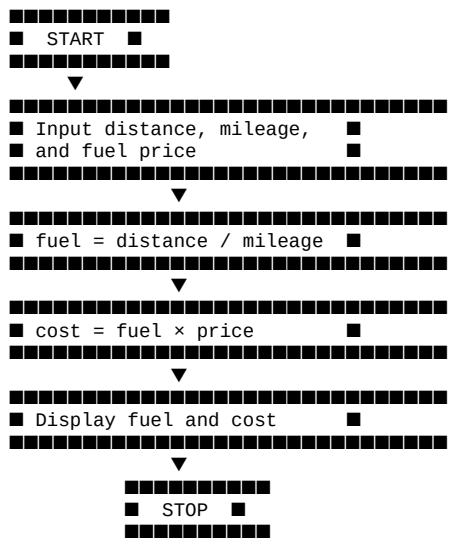
3. Calculate fuel = distance / mileage.

4. Calculate cost = fuel × price.

5. Display fuel required and total cost.

6. Stop.

5. Flowchart



6. Pseudocode

```
BEGIN
    INPUT distance, mileage, price
    fuel = distance / mileage
    cost = fuel * price
    PRINT fuel, cost
END
```

7. Python Source Code

```
distance = float(input("Enter journey distance in km: "))
```

```
mileage = float(input("Enter vehicle mileage in km/litre: "))
price = float(input("Enter fuel price per litre: "))

fuel = distance / mileage
cost = fuel * price

print("Fuel Required:", fuel, "litres")
print("Total Fuel Cost: Rs.", cost)
```

8. Test Cases

Distance	Mileage	Price/litre	Expected result
200	20	100	10 litres; ■1000
150	15	100	10 litres; ■1000

9. Sample Output

```
Enter journey distance in km: 200
Enter vehicle mileage in km/litre: 20
Enter fuel price per litre: 100
Fuel Required: 10.0 litres
Total Fuel Cost: Rs. 1000.0
```

10. Conclusion

The program uses arithmetic operations to estimate fuel consumption and journey cost.

Case Study 27: Advanced Numeric Pattern Generator

1. Problem Statement

Write a Python program using nested loops to print a right-angled number pattern for a user-specified number of rows.

2. Objectives

• Accept the number of rows.

• Use nested loops to generate the pattern.

• Print an increasing sequence of numbers on each row.

3. Assumptions

• The number of rows is a positive integer.

• The pattern begins at 1 on every row.

• This is a simple numeric pattern from the broader pattern-generation case study.

4. Algorithm

1. Start.

2. Read the number of rows n.

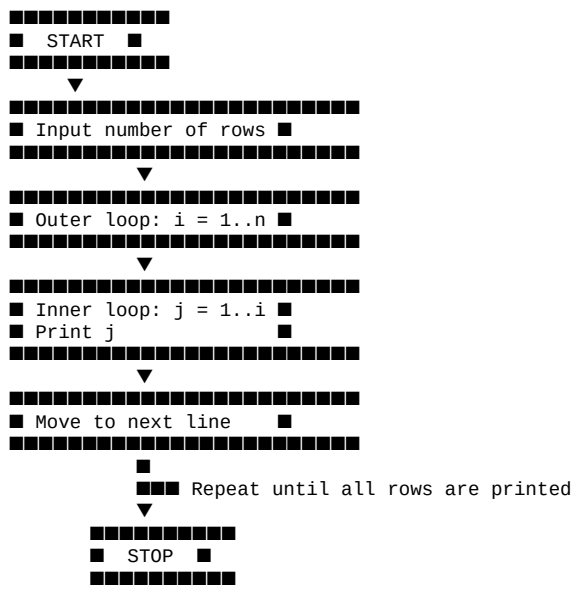
3. Repeat i from 1 through n.

4. For each row, repeat j from 1 through i and print j on the same line.

5. Move to a new line after each row.

6. Stop.

5. Flowchart



6. Pseudocode

```
BEGIN
    INPUT n
    FOR i = 1 TO n
        FOR j = 1 TO i
            PRINT j without newline
        ENDFOR
        PRINT newline
    ENDFOR
END
```

7. Python Source Code

```
n = int(input("Enter number of rows: "))

for i in range(1, n + 1):
    for j in range(1, i + 1):
        print(j, end=" ")
    print()
```

8. Test Cases

Rows	Expected output
3	1 / 1 2 / 1 2 3
5	Rows contain 1; 1 2; 1 2 3; 1 2 3 4; 1 2 3 4 5

9. Sample Output

```
Enter number of rows: 5
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

10. Conclusion

The program demonstrates nested loops and output formatting to generate a simple number pattern.

Case Study 29: Password Policy Validation Engine

1. Problem Statement

Write a Python program to check whether a password contains at least eight characters, at least one digit, and at least one uppercase letter.

2. Objectives

• Read a password string.

• Check the password length.

• Check for a digit and an uppercase letter.

• Display the first rule that is not satisfied or report a valid password.

3. Assumptions

• The minimum password length is eight characters.

• At least one digit and one uppercase letter are required.

• This simplified exercise does not check symbols, repeated characters, or prohibited words.

4. Algorithm

1. Start.

2. Read the password.

3. If its length is less than 8, display a length warning.

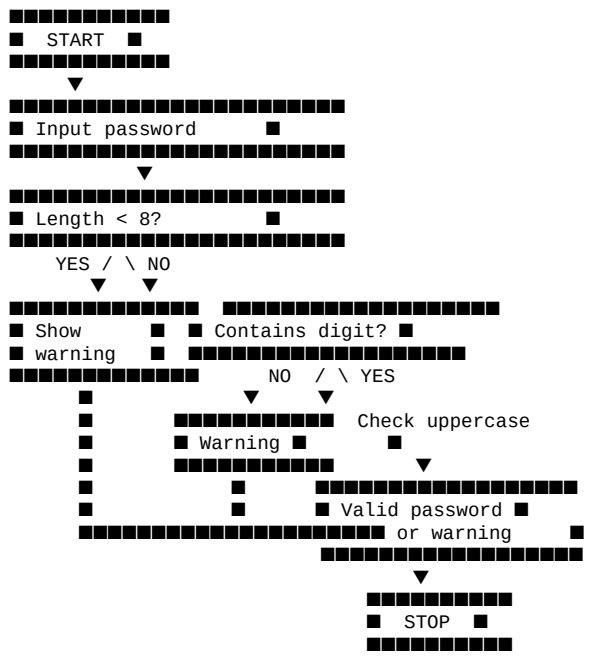
4. Otherwise, if no digit is present, display a digit warning.

5. Otherwise, if no uppercase letter is present, display an uppercase warning.

6. Otherwise display Valid Password.

7. Stop.

5. Flowchart



6. Pseudocode

```
BEGIN
    INPUT password
```

```
    IF length(password) < 8 THEN
        PRINT "Password must contain at least 8 characters"
    ELSE IF password has no digit THEN
        PRINT "Password must contain a number"
    ELSE IF password has no uppercase letter THEN
        PRINT "Password must contain an uppercase letter"
    ELSE
        PRINT "Valid Password"
    ENDIF
END
```

7. Python Source Code

```
password = input("Enter password: ")

if len(password) < 8:
    print("Password must contain at least 8 characters")
elif not any(ch.isdigit() for ch in password):
    print("Password must contain a number")
elif not any(ch.isupper() for ch in password):
    print("Password must contain an uppercase letter")
else:
    print("Valid Password")
```

8. Test Cases

Password	Expected result
Hello123	Valid Password
hello	Password must contain at least 8 characters
helloworld	Password must contain a number
hello1234	Password must contain an uppercase letter

9. Sample Output

```
Enter password: Hello123
Valid Password
```

10. Conclusion

The program uses string methods, len(), any(), and conditional statements to check three basic password rules.